

An AI-Powered Technical Debt Analytics Dashboard for Agile Small-Scale Software Teams

Feasibility Report

Group – 16

Project ID – 7

CS3203 – Software Engineering Project

Mentor: Mr. Anju Chamantha

Name	Index number	Contribution
Nethmini R.M.N.	230432G	Resource and time feasibility, Social / Legal feasibility and Functional considerations.
Nethsara U.D.K.C.	230435T	Technical feasibility study, non-functional considerations and project schedule
Pabasara H.H.J.	230451M	Financial feasibility, project schedule and Risk feasibility.

Date: 12th July, 2026

Table of Contents

1. Introduction.....	1
1.1 Overview of the Project	1
1.2 Objectives of the Project.....	1
1.3 The Need for the Project	1
1.4 Overview of Existing Systems and Technology	1
1.4.1 Existing Techniques	1
1.4.2 Existing systems and tools	2
SonarQube.....	2
CodeScene.....	2
CodeRabbit	2
1.4.3 Proposed System Implementation.....	2
1.5 Scope of the Project	3
1.5.1 Core Functionalities:	3
1.5.2 Out of Scope:	3
1.6 Deliverables	3
2. Feasibility Study	4
2.1 Financial Feasibility.....	4
2.1.1 Development Cost (One Time)	4
2.1.2 Estimated Monthly Operational & Maintenance Cost (Fixed)	5
2.1.3 Customer ROI Value	6
2.1.4 Software Pricing.....	6
2.1.5 Estimated Gross Profit Margin (Per user/month)	7
2.1.6 Financial Feasibility Evaluation	7
2.2 Technical Feasibility	7
2.2.1 Proposed Technologies	8
2.2.2 Technical Feasibility Evaluation.....	9
2.3 Resource and Time Feasibility.....	9
2.3.1 Project schedule and key Milestones	9
2.3.2 Resource Requirements	11
2.4 Risk Feasibility	11
2.5 Social/Legal Feasibility	12
2.5.1 Social feasibility:	12

3. Considerations.....	13
3.1 Functional considerations	13
3.2 Non-functional considerations	14
4. References	15

List of Tables

Table 1: Proposed Technology Stack and Resources for System Components	3
Table 2: Estimated One-Time Development Cost	4
Table 3: Estimated Monthly Operational and Maintenance Cost	5
Table 4: Project Schedule and Key Milestones.....	10
Table 5: Project Risk Assessment and Mitigation Strategies.....	11

1. Introduction

1.1 Overview of the Project

The project proposes an AI-Powered Technical Debt Analytics Dashboard for small-scale agile software teams; delivered as multi-tenant web-based SaaS platform. The platform allows the teams to identify, analyze and prioritize technical debt. Teams connect their GitHub/GitLab repositories, and the system will analyze the source code, code comments, and commit messages of the selected repository to identify different types of technical debt using static analysis and machine learning. It will output a code health score, categorize the identified technical debt and prioritize the issues. Unlike existing tools, the platform consists of cross repository analysis, organization configurable profiles. This allows developers to understand the current condition of their codebase and identify the areas that require immediate attention.

1.2 Objectives of the Project

- Analyze direct impacts of technical debt to the velocity of small-scale agile teams.
- Design and implement a secure, multi-tenant web architecture for analyzing code repositories.
- Provide a centralized, interactive dashboard visualizing technical debt metrics in a single interface.
- Develop and train an NLP-driven Machine Learning module capable of classifying source code comments and commit messages into debt categories.
- Evaluate the accuracy of ML-driven bug prediction algorithms compared to traditional, rule based static code analysis.

1.3 The Need for the Project

Small and medium scale agile software teams work under tight deadlines and limited resources often leading to bad smells in code. These firms often lack Senior software engineers' reviews, so more bad smell produced by less experienced developers often go unnoticed. Recent research found that the developers spent roughly 23% of their time on technical debt which will be a major pain point for small agile teams due to lack of staff on manual code reviews [1]. Combined with all, the overall project velocity reduces unnoticeably.

Yet there are numerous tools for code reviewing, small agile teams struggle to adapt them as they are built for large organizations which have dedicated teams for configuration and code reviews. It is rarely found in small agile teams with diverse engineering backgrounds; Therefore, these systems make high overhead in configuration and adapting the software. On the other hand, above systems mostly rely on static deterministic rules, resulting in hundreds of issues making it unsuitable for small agile teams. Rather than having unnecessary noise and complex configurations, these teams prefer simple tools which only prioritize the most critical issues that need immediate fixing. In agile principal teams prefer adding new features in each increment, therefore fixing low priority technical debt has no visible customer value.

1.4 Overview of Existing Systems and Technology

1.4.1 Existing Techniques

Static code analysis is a fundamental software quality assurance technique that involves systematically inspecting source code without executing it aiming to detect bugs, vulnerabilities, code smells and compliance issues and it helps minimize and address technical debt [2]. To identify and prioritize the identified technical debt, some tools use Behavioral Code Analysis. Behavioral Code Analysis identifies patterns in how a development organization interacts with the code base they are building. This analysis is used to prioritize technical debt, detect implicit dependencies that are invisible in the code itself and measure organizational factors like knowledge gaps and support on and off boarding [3]. With the advancement of Artificial Intelligence, tools use AI-powered static code analysis to identify problem areas. Also, they analyze historical data, learning from past coding patterns and their outcomes so that the analysis can alert teams to potential debt-ridden areas in codebase, even before they become a significant problem [4].

1.4.2 Existing systems and tools

SonarQube [5]

SonarQube is a self-managed, enterprise-grade and static analysis based continuous code quality and static application security testing platform that supports more than thirty programming languages [6]. However, it requires considerable configuration effort for small development teams and generates a large number of findings that require manual filtering.

CodeScene [7]

CodeScene is a platform which uses behavioral code analysis to identify and prioritize technical debt of the software products and it outputs prioritized hotspot maps, numeric code health scores, financial cost impact trends, and automated delivery risk classifications [8]. Even though it provides powerful organizational insights, it cannot output actionable results for codebases of a small-scale team.

CodeRabbit [9]

CodeRabbit is an AI-powered code review platform that analyzes every code change throughout the pull request lifecycle. It provides automated reviews, identify potential bugs, security vulnerabilities, code quality issues and suggests possible fixes [4]. As it scans the code only at each increment, it does not allow to detect where older technical debt exists or to track long-term repository health across the entire project lifecycle.

1.4.3 Proposed System Implementation

The proposed solution combines lightweight static code analysis with machine learning techniques to provide actionable technical debt prioritization for small-scale agile software teams instead of generating extensive lists of code quality issues. The source code is analyzed using static analysis to extract software quality metrics while code comments and commit messages are analyzed in parallel using NLP-based Self-Admitted Technical Debt (SATD) classifier. The extracted software metrics are used by a bug prediction model to estimate the likelihood of future defects. Then, the outputs of the static analysis, SATD classifier and bug prediction model are integrated into a configurable technical debt score that ranks source files according to their refactoring priority. The results are displayed on the interactive dashboard.

Table 1: Proposed Technology Stack and Resources for System Components

Component	Task	Technology/ Resource
Frontend	Analytics dashboard	Next.js [10]
Backend	Repository integration and API	FastAPI [11]
Static Analysis	Extract software metrics	Lizard [12]
ML Module	SATD detection and classification	Python [13], Scikit-Learn [14], SATD Dataset [15]
	Bug proneness prediction	Python [13], Scikit-Learn [14], GHPR dataset [16], NASA PROMISE dataset [17]
Database	Store analysis results	PostgreSQL [18]

1.5 Scope of the Project

1.5.1 Core Functionalities:

- Connect repository with GitHub App/ Integrate with GitHub Actions workflows/ or paste a URL directly.
- Cross repository dependency analytics across workspace repositories.
- Run analysis with asynchronous worker queues to keep UI responsive.
- Support silent checks (silent check is a background check automatically triggered on every push or pull request) to auto refresh dashboard.
- Detection of security related technical debt (security keyword SATD, hardcoded secrets, SQL string concatenation, dangerous eval/exec).
- Provide a dashboard with overall code health with interactive file tree view to show hotspots in code.
- Produce prioritized list of “fix first” issues using organization configurable weighting profiles.

1.5.2 Out of Scope:

- Fully automatic code refactoring service
- Support for proprietary/on-premise distributed version control system beyond GitHub/GitLab
- Integration with Project management tools such as Jira/Trello
- Full vulnerability scanning (SAST/DAST, CVE/dependency auditing, penetration testing).

1.6 Deliverables

- **A web-based software system (SaaS):**
A multi-tenant system where a user connects their GitHub/ GitLab repository to receive a health report & Commit graph with refreshable Dashboard
- **An interactive Graphical User Interface (GUI):**
A user-friendly and interactive dashboard which displays the debt analysis outcomes clearly to identify easily using prioritizing according to user preferences.

- **A Machine Learning Model:**
A Python based trained machine learning model automatically scans code comments and commits messages for debt classification. (documented precision/recall metrics for debt classification)
- **System architecture documentation:**
Complete technical documentation explained the interaction between the backend and the ML service.
- **A project report:**
A final detailed report which validates the tool's utility by analyzing the real-world existing repositories.

2. Feasibility Study

2.1 Financial Feasibility

The financial feasibility of the proposed SaaS platform is analyzed using a cost-benefit model. The proposed pricing strategy is derived by considering multiple parameters: estimated gross profit margin, customer ROI (Return On Investment) and break-even analysis. The development effort, maintenance and customer support are cost free. As the development effort is contributed by the project team without financial contribution and all required development tools are freely available, no direct development, maintenance, or customer support costs are involved during the project period. Therefore, only operational infrastructure costs are considered in this financial feasibility analysis.

Assumptions

1. Fixed Infrastructure Capacity: The first release is scaled for 50 teams of 5 developers.
2. Typical agile team: 5 developers with an average salary of USD 1,000/month per developer.

2.1.1 Development Cost (One Time)

The one-time cash total is effectively zero when excluding the internal development effort.

Table 2: Estimated One-Time Development Cost

Cost Item	Cost Driven by	Cost Estimation
Development effort	Design, implementation, validation, and documentation	USD 0

ML Training	Training for SATD & bug proneness models	USD 0 (Free tiers of Google Colab [19] / Kaggle [20])
Datasets	SATD Dataset [15], GHPR [16], NASA PROMISE [17]	USD 0 (free public datasets)
IDEs & Dev tools	VS Code, Git, Docker	USD 0 (Free/Open source)
Source code hosting / CI	GitHub	USD 0
Design diagrams & documentation	draw.io, LucidChart, Mermaid, MS Office	USD 0
Total one-time cost		USD 0

2.1.2 Estimated Monthly Operational & Maintenance Cost (Fixed)

All operational and maintenance infrastructure costs are estimated using standard pricing and breakdown from cloud platforms such as Digital Ocean [21] [22].

Table 3: Estimated Monthly Operational and Maintenance Cost

Cost Item	Cost Driven by	Cost Estimation
Frontend hosting	Next.js dashboard	USD 5
Backend hosting	FastAPI	USD 12
Worker service Message Broker	Asynchronous repository analysis jobs	USD 8
PostgreSQL	Multi-tenant database	USD 20
ML inference	Batch inference	USD 10
Domain + TLS	SaaS delivery	USD 1

Monitoring, logging & uptime	Operations	USD 4
Backups	Data protection	Included in Database
Maintenance	Updates and bug fixing	USD 0
Periodic ML model re-training	Model refresh on new data	Included in maintenance
Total Fixed Operational Cost		USD 60

2.1.3 Customer ROI Value

Research shows that developers waste approximately 23% of their time on technical debt [1]. For an assumed agile team of 5 developers earning USD 1,000/month each, the cost of this wasted time is USD 1,150 per month ($5 \times \$1,000 \times 0.23$).

If the proposed system can reduce this waste by just 5% using its prioritized list, the organization saves USD 250 per month ($5 \times \$1,000 \times 0.05$). If the monthly subscription plan is priced at USD 50 (USD 10 per user $\times$ 5 users), the team saves 5x the value of the subscription cost.

2.1.4 Software Pricing

Competitor Pricing

- **SonarQube Cloud:** Paid Team plans are priced at USD 20 and USD 40 per user/month based on features. They also offer a separate team plan for USD 34 per month with a limited line-of-code (LOC) allowance [23].
- **CodeScene:** The Standard tier is priced at USD 21 per user/month, while the Pro tier also costs USD 21 per user/month [24].

Value based pricing

With USD 200/month of recoverable value per team of 5 developers, a price of USD 50/month per team offers the customer a significant share of the value created by SaaS software.

Selected Pricing Structure

- **Free:** USD 0 (Unlimited scanning of public repositories, single workspace).

- **Team:** USD 10 per user/month (Private repositories, multi-repository workspaces, configurable prioritization, silent checks).

2.1.5 Estimated Gross Profit Margin (Per user/month)

Variable cost per paid user: Estimated cost of USD 1/month (covers payment processing [25], additional cloud infrastructure usage, and customer support).

Gross profit margin:

$$\frac{(\text{Price} - \text{Variable cost})}{\text{Price}} \times 100 = \frac{(10 - 1)}{10} \times 100 = 90\%$$

This indicates that the proposed SaaS product can expect highly profitable margins, easily competing with typical SaaS industry margins of 70% to 90% [26].

Break-even Point

Minimum paid users to cover operational costs:

$$\frac{\text{Fixed monthly operational cost}}{(\text{Price per user} - \text{Variable cost per user})}$$

Calculation:

$$\frac{60}{(10 - 1)} = 6.7 \text{ approximately } 7 \text{ paid users}$$

Therefore, acquiring 2 teams of 5 developers is enough to cover operational costs.

2.1.6 Financial Feasibility Evaluation

Initial development requires no external funding. All necessary tools, training datasets, and training infrastructure are freely available. After launch, fixed operational costs can be completely covered by just 7 users. Furthermore, the combination of a 90% gross profit margin and a 5x customer ROI strongly suggests that the proposed system is highly financially feasible to implement.

2.2 Technical Feasibility

To provide a prioritized technical debt analysis for small-scale agile software teams, the proposed system combines several components each responsible for a specific function. These components

are built on web development frameworks, static source code analysis tools, machine learning libraries, version control platform APIs and using publicly available resources.

2.2.1 Proposed Technologies

Frontend

Next.js [10] has been chosen for the development of the frontend of the system which acts as the interface between the users and the system. It is a flexible React framework that provides building blocks for developing fast and scalable web applications [27]. It is a mature framework which is actively maintained and widely used by organizations of various sizes worldwide [28] [29]. It is well suited for implementing the interactive dashboard of the system as it supports component-based development, efficient routing and responsive user interfaces [2]. Tailwind CSS [30] will be used to achieve rapid development of the user interface keeping it responsive and consistent.

Backend (and workers)

The backend is the major processing component of the system. It executes the business logic, communicates with the frontend and interacts with the external services like GitHub and GitLab. It also coordinates static analysis and modules and stores the results in the database. FastAPI [11] is selected as the backend framework of the system. It is a modern, high-performance Python web framework designed to build RESTful APIs with automatic interactive documentation and asynchronous request handling. Celery [31] will be used to execute repository analysis and machine learning inference asynchronously and Redis [32] will support efficient background processing as the message broker without blocking user requests.

Machine Learning & Static Analysis

To identify self-admitted technical debt (SATD) as well as to predict bug prone code components a machine learning component is used in the system. To support the predictions, software quality metrics are extracted by performing static code analysis. These metrics serve as an important input for the machine learning models as they provide quantitative measurements of code quality. Lizard is selected as the static code analysis tool for this because it is a lightweight, open-source application that can extract software complexity metrics from many programming languages [12].

Python [13] and Scikit-learn [14] are chosen to develop the machine learning models. Scikit-learn is an open-source machine learning library and provides a wide range of supervised learning algorithms, preprocessing utilities and model evaluation techniques. Hence, Scikit-learn has the necessary support to develop, train, evaluate and compare multiple models for this component of the system as it performs supervised learning tasks such as SATD classification and bug proneness prediction using labelled public datasets. Publicly available datasets such as the SATD Dataset [15], and NASA PROMISE Dataset [17] have labelled software engineering data that is required for training and evaluating the machine learning models of the system.

Database

The database is used for storing tenant information, user accounts, repository metadata and historical analysis results required for visualization and trend analysis. It should maintain the logical isolation between tenants to ensure that each tenant can access its own results only. PostgreSQL is an open-source relational database management system that supports multiple multi-tenancy models and is well-suited for SaaS application. It provides reliable transactional

support, extensibility and features such as Row Level Security (RLS) which can be used to centrally enforce tenant data isolation within shared database environments [33].

Repository Integration

The system will use GitHub [34] and GitLab [35] official APIs to access the repositories. For GitHub, GitHub Apps will be used as it provides secure installation-based authentication with fine-grained repository permissions [36]. These well-documented interfaces allow secure retrieval of repository metadata, source code, commit histories of both private and public repositories and adhere to the principle of least privilege.

Deployment

The system will be deployed in a cloud environment to support its Software-as-a-Service architecture. They provide scalable computing resources, managed networking and cloud services that are required to host all the components of the system. Their scalability and high availability support multiple tenants and increasing repository analysis workloads. The application components will be containerized using Docker to package the components with their dependencies, improving portability and enabling efficient management of the cloud-hosted application [37].

2.2.2 Technical Feasibility Evaluation

The proposed system is considered technically feasible based on the analysis of the proposed technologies. The selected technologies are mature, actively managed and widely used within industry. The availability of well-established web frameworks, static code analysis tools, machine learning libraries, secure repository integration mechanisms and publicly available datasets have reduced the technical risks associated with the project significantly. Hence, no major technical barriers which would prevent successful development of the proposed system have not been identified within the planned project scope.

2.3 Resource and Time Feasibility

The project is capable of completing within the given time period using the available resources. The project team consists of three members working together and the workload is shared equally to make sure any team member won't be exhausted with workload. Each member will work on separate modules at the same time to speed up the development process and to make sure the validation of the project before the project is being integrated each member will run a collaborative code review.

2.3.1 Project schedule and key Milestones

A complete plan has been created by the development team using a Gantt chart to track the progress. The team tries to finish tasks as early as possible by making a buffer window. So, if any unforeseen problem happens like illness still the project is capable of completing within the given time.

Table 4: Project Schedule and Key Milestones

Phase Number	Task	Description of key activities	Start Date	End Date
1	Project Proposal	Problem identification, literature review, project objective definition, proposal writing.	Jun 30	Jul 5
2	Feasibility Study & Project Schedule	Feasibility analysis, project planning, Gantt chart preparation, feasibility report writing.	Jul 6	Jul 12
3	Requirements Engineering & SRS	Use cases, functional/non-functional requirements, dashboard output definitions	Jul 13	Aug 2
4	Architecture & System Design Document	Component architecture, Open API contracts, multi-tenant DB schema, UI wireframes	Jul 23	Aug 7
5	Environment & Background Setup	Repos, Docker Compose, CI pipeline, deployed	Jul 20	July 25
6	Multi-tenancy, Auth & RBAC	Tenant/user models, roles, Row-Level Security policies	Aug 4	Aug 9
7	Dashboard Completion	Layout, navigation, health-score and file-tree visualization	Aug 7	Aug 13
8	ML Data Preparation & Baselines	Download and clean datasets, feature extraction; train/test splits; TF-IDF + SVM baseline	Aug 5	Aug 19
9	Repository Integration	Public-URL clone path, GitHub App installation flow, webhooks, least-privilege access handling	Aug 14	Aug 20
10	Metric Extraction & Rule Engine	Lizard metrics, git history mining, duplication, code smell thresholds and Celery scan pipeline	Aug 19	Aug 26
11	ML Model Development	SATD model development using Scikit-learn, bug-proneness model development, evaluation and model export	Aug 27	Sep 9
12	Prioritization Engine	Health-score aggregation, Refactor-First ranking, configurable profile weight vectors	Aug 28	Sep 12

13	ML Service Integration	Deploy ML microservice, connect workers to inference, score explanations	Sep 10	Sep 15
14	System Testing & Evaluation	Integration/E2E tests, tenant-isolation tests, ML vs rule-baseline evaluation on real repositories, bug fixing	Sep 15	Sep 23
15	Testing Report	Document test cases, execution results, identified defects, ML evaluation metrics, and overall system testing analysis.	Sep 23	Sep 26
16	Hardening & Release	Deployment hardening, final fixes, production release	Sep 24	Sep 30

2.3.2 Resource Requirements

The project will be developed with the personal laptops of the development team using GitHub for version control, collaboration and project management while Google docs and other online collaboration tools will support documentation. The team will utilize official documentation, research publications and other publicly available resources to study technical debt analysis techniques and implement the solution effectively. As discussed in the financial and technical feasibility sections the required software tools, technologies and infrastructure are already available which ensure the resources needed to complete the project successfully are adequate and readily available [38].

2.4 Risk Feasibility

Risks were identified and assessed by following the risk management process. Risks are categorized by risk type, with probability rated on Sommerville's scale [39]. (low, moderate, high) and effects rated as catastrophic, serious, tolerable or insignificant.

Table 5: Project Risk Assessment and Mitigation Strategies

No	Category	Risk	Probability	Severity	Mitigation Strategy
1	People	Developers are ill and unavailable at critical times	Moderate	Catastrophic	Shared code ownership; GitHub issues & GitHub projects to schedule task; buffer weeks

2	Estimation	The time required to develop the software is underestimated	High	Serious	Buffered Gantt schedule; weekly progress tracking; core features built first
3	Technical	ML models underperform	Moderate	Serious	Per-repository calibration; rule-based engine delivers can run standalone
4	Technical	GitHub API rate limits	Moderate	Tolerable	Conditional requests with ETag caching; GitHub App installation tokens [40] .
5	Resource	Free-tier limits exhausted (hosting hours, database storage, Actions minutes, Colab GPU quota)	High	Tolerable	Weekly usage monitoring; workload spread across providers.
6	Technical	Model accuracy drifts over time	High	Tolerable	Periodic re-training on recent data; re-training supported model design; accuracy monitoring.
7	Requirements	Scope drifts away from proposed SaaS product	Low	Serious	Scope locked in proposal; new ideas listed under future improvements list.

2.5 Social/Legal Feasibility

The project is both socially and legally feasible as it does not violate user privacy regulations. It provides a positive value to its user.

2.5.1 Social feasibility:

Social feasibility involves evaluating the project on the people and society ensuring the project will be completely accepted and supported by the community [38].

- **Public acceptance:** The project design is highly user friendly and trustable as the project guarantees for what the system uses user repositories and unlike the current technical debt analyzing system the project has a simpler and more understandable dashboard removing the extra noise.
- **Cultural norms:** The system solves real world problems which small agile teams often face without violating any cultural or social norms.
- **Ethical impacts:** The project mainly focused on small agile teams without any bias or harm to society and does not harm any social ethic.

Legal Feasibility:

Legal feasibility refers to assessment of whether the project complies with the laws and regulations governing its operation ensuring the project can operate within the legal framework [41].

- **Structure:** The project operates under standard university project guidelines and follows well-defined professional engineering structures.
- **Daily rules:** The project operates strictly under standard privacy policy and data protection regulation. All data stored in PostgreSQL database is securely handled and no unauthorized data sharing takes place.
- **Licensing:** The project uses freely available development utilities which protect intellectual property rights. The team follows all open-source licensing agreements ensuring zero copyright violations.

3. Considerations

3.1 Functional considerations

- **Automatic Repository Scanning:** The system capable of establishing a secure connection via GitHub Apps or GitLab API to automatically scan the code repositories without requiring user manual code upload.
- **Technical Debt Categorization and Detection:** The system have the ability to reliably extract code metrics to detect specific types of technical debt and should categorize them.
- **AI-Driven Bug Proneness Prediction:** The machine learning pipeline analyses historic repo patterns and software metrics to flag files or blocks of code that have a high probability of introducing runtime defects.
- **Actionable Prioritization Logic:** The dashboard prioritizes the detected technical debt and also provides sorting and filtering features according to user preferences to avoid small teams being overwhelmed with an unmanageable refactoring list.

- **Agile Workflow Integration:** The system visualizes the technical debt trends, historical comparisons and sprint over sprint progress supporting development teams to monitor and manage technical debt throughout the development process.

3.2 Non-functional considerations

Performance

The analytics dashboard should be responsive while repository analysis is performed in the background. This is feasible because FastAPI supports asynchronous request handling and Celery workers can be used to process computationally heavy analysis tasks independently from the user interface.

Security

The system should protect user data and repository privacy by adhering to secure authentication and access control mechanisms. This is feasible as GitHub Apps and GitLab's secure authentication mechanisms with fine-grained, read-only repository permissions. Users should explicitly choose which repositories the system is authorized to access. Hence, it can only read the selected repositories required for analysis and cannot access, read or modify any other private or public repositories.

The multi-tenant database is designed to logically isolate each tenant's data using PostgreSQL Row-Level Security and ensures that users from accessing analysis results or organizational data that belong to other tenants. All communication is encrypted using HTTPS and the sensitive information is securely stored within the database.

Usability

The users should be able to identify high-priority technical debt using the system minimizing the effort required. This is feasible through the dashboard that highlights the most critical technical debt items using AI-driven prioritization, reducing the need for manual configurations and it does not overwhelm the users by showing numerous technical debt issues and software quality metrics.

Scalability

The system should efficiently support multiple tenants, repositories and concurrent analysis requests with the growth of the number of users. This is feasible through the multi-tenant architecture, cloud deployment and containerized services that allows individual components like backend, database and background workers to be scaled independently according to varying demand.

4. References

- [1] T. Besker, A. Martini and J. Bosch, "Technical Debt Cripples Software Developer Productivity," 2018.
- [2] Sonar, "Static Code Analysis Using SonarQube Community Build: A Step-by-Step Guide," Sonar, [Online]. Available: <https://www.sonarsource.com/resources/library/static-code-analysis-using-sonarqube/>. [Accessed 1 July 2026].
- [3] Adam Tornhill, "Better than silver bullets: A milestone for behavioral code analysis," CodeScene, [Online]. Available: <https://codescene.com/blog/validation-for-behavioral-code-analysis/>. [Accessed 1 July 2026].
- [4] CodeRabbit, "Reduce Tech Debt: AI's Role in Efficient Coding," CodeRabbit, [Online]. Available: <https://www.coderabbit.ai/blog/reduce-tech-debt-ai-s-role-in-efficient-coding>. [Accessed 1 July 2026].
- [5] Sonar, "SonarQube - Code verification for the AI era," Sonar. [Online]. [Accessed 1 July 2026].
- [6] Sonar, "Static Code Analysis Using SonarQube Community Build : A Step-by-Step Guide," 2026. [Online]. Available: <https://www.sonarsource.com/resources/library/static-code-analysis-using-sonarqube/>. [Accessed 1 July 2026].
- [7] CodeScene, "Scale AI Coding Safely and Mitigate Technical Debt," CodeScene. [Online]. [Accessed 1 July 2026].
- [8] CodeScene, "Behavioral Code Analysis in Practice," CodeScene, 2023.
- [9] CodeRabbit, "AI Code Reviews | CodeRabbit," CodeRabbit, [Online]. Available: <https://www.coderabbit.ai/>. [Accessed 2 July 2026].
- [10] Vercel, "Next.js by Vercel - The React Framework," Vercel, [Online]. Available: <https://nextjs.org/>. [Accessed 2 July 2026].
- [11] FastAPI, "FastAPI," [Online]. Available: <https://fastapi.tiangolo.com/>. [Accessed 2 July 2026].
- [12] T. Yin, "Lizard," [Online]. Available: <https://github.com/terryyin/lizard/blob/master/README.rst>. [Accessed 2 July 2026].
- [13] Python Software Foundation, "Welcome to Python.org," Python Software Foundation. [Online]. [Accessed 2 July 2026].
- [14] scikit-learn, "scikit-learn: machine learning in python," scikit-learn, [Online]. Available: <https://scikit-learn.org/stable/>. [Accessed 2 July 2026].

- [15] Y. Li, M. Soliman and P. Avgeriou, "Automatic identification of self-admitted technical debt from four different sources," *Empirical Software Engineering*, vol. 28, no. 65, 2023.
- [16] J. Xu, F. Wang and J. Ai, "Defect Prediction With Semantics and Context Features of Codes Based on Graph Representation Learning," *IEEE Transactions on Reliability*, 2021.
- [17] ApoorvaKrisna, "NASA Promise Dataset Repository," 2020. [Online]. Available: <https://github.com/ApoorvaKrisna/NASA-promise-dataset-repository>. [Accessed 4 July 2026].
- [18] The PostgreSQL Global Development Group, "PostgreSQL: The world's most advanced open source database," The PostgreSQL Global Development Group, 2026. [Online]. Available: <https://www.postgresql.org/>. [Accessed 1 July 2026].
- [19] Google, "Colaboratory - Frequently Asked Questions," [Online]. Available: <https://research.google.com/colaboratory/faq.html>. [Accessed 11 July 2026].
- [20] Kaggle, "Notebook Documentation," [Online]. Available: <https://www.kaggle.com/docs/notebooks>. [Accessed 11 July 2026].
- [21] DigitalOcean, "Pricing," [Online]. Available: <https://www.digitalocean.com/pricing>. [Accessed 11 July 2026].
- [22] DigitalOcean, "Pricing Calculator," [Online]. Available: <https://www.digitalocean.com/pricing/calculator>. [Accessed 11 July 2026].
- [23] Sonar, "Plans and Pricing," [Online]. Available: <https://www.sonarsource.com/plans-and-pricing/>. [Accessed 11 July 2026].
- [24] CodeScene, "Pricing," [Online]. Available: <https://codescene.com/pricing>. [Accessed 11 July 2026].
- [25] Stripe, "Pricing and Fees," [Online]. Available: <https://stripe.com/pricing>. [Accessed 11 July 2026].
- [26] Stripe, "SaaS Gross Margin Explained," [Online]. Available: <https://stripe.com/resources/more/saas-gross-margin-explained-what-it-is-and-why-it-is-important>. [Accessed 11 July 2026].
- [27] Vercel, "React Foundations: About React and Next.js | Next.js," 2026. [Online]. Available: <https://nextjs.org/learn/react-foundations/what-is-react-and-nextjs#building-blocks-of-a-web-application>. [Accessed 5 July 2026].
- [28] Vercel, "Showcase | Next.js by Vercel - The React Framework," Vercel, [Online]. Available: <https://nextjs.org/showcase>. [Accessed 5 July 2026].
- [29] Vercel, "next.js," vercel, [Online]. Available: <https://github.com/vercel/next.js>. [Accessed 5 July 2026].

- [30] tailwindcss, "Tailwind CSS - Rapidly build modern websites without ever leaving your HTML," 2026. [Online]. Available: <https://tailwindcss.com/>. [Accessed 3 July 2026].
- [31] A. Solem, "Workers Guide - Celery 5.6.3 documentation," 2023. [Online]. Available: <https://docs.celeryq.dev/en/stable/userguide/workers.html>. [Accessed 5 July 2026].
- [32] Redis, "Docs," Redis, 2026. [Online]. Available: <https://redis.io/docs/latest/>. [Accessed 3 July 2026].
- [33] Michel Beardsley, "Multi-tenant data isolation with PostgreSQL Row Level Security | AWS Database Blog," 18 May 2020. [Online]. Available: <https://aws.amazon.com/blogs/database/multi-tenant-data-isolation-with-postgresql-row-level-security/>. [Accessed 5 July 2026].
- [34] GitHub, "GitHub," GitHub, [Online]. Available: <https://github.com/>. [Accessed 6 July 2026].
- [35] GitLab, "Finally AI for the entire software," GitLab, 2026. [Online]. Available: <https://about.gitlab.com/>. [Accessed 6 July 2026].
- [36] GitHub, "About creating GitHub Apps," 2026. [Online]. [Accessed 6 July 2026].
- [37] Docker, "Why Docker," 2026. [Online]. Available: <https://www.docker.com/why-docker/>. [Accessed 6 July 2026].
- [38] GeeksforGeeks, "Types of Feasibility Study in Software Project Development," 3 October 2024. [Online]. Available: <https://www.geeksforgeeks.org/software-engineering/types-of-feasibility-study-in-software-project-development/>. [Accessed 11 July 2026].
- [39] I. Sommerville, Software Engineering, 10 ed., Harlow: Pearson Education, 2016.
- [40] GitHub, "Rate limits for the REST API," [Online]. Available: <https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api>. [Accessed 11 July 2026].
- [41] Studocu, "Differentiate between legal and social feasibility," 2026. [Online]. Available: <https://www.studocu.com/row/messages/question/11524067/differentiate-between-legal-and-social-feasibility>. [Accessed 11 July 2026].
- [42] OSWAP Foundation, "Static Code Analysis," [Online]. Available: https://owasp.org/www-community/controls/Static_Code_Analysis. [Accessed 30 June 2026].
- [43] CodeScene, "Behavioral Code Analysis in Practice," CodeScene, 2023.
- [44] Codacy, "Official information about Codacy," 27 May 2026. [Online]. Available: <https://www.codacy.com/ai-info#basic-information>. [Accessed 1 July 2026].
- [45] Codacy, "Code Quality for AI-Assisted Engineering Teams | Codacy," Codacy. [Online]. [Accessed 1 July 2026].